

Artificial Intelligence & Data Engineering
Design I
Interim Report

Title

**Advanced Model Compression Techniques for
Resource-Constrained AI Architectures**
**Does $W+W^T$ Weight Sharing Generalize to Language
Models?**

Prepared By

[Student ID] Alper Düzgün
[Student ID] Hazar Utku Sözer

Supervisor

Prof. Dr. Behçet Uğur Töreyn

June, 2026

İTÜ



250 YIL
1773 – 2023

CONTENTS

1. INTRODUCTION.....	5
1.1. MOTIVATION AND PROBLEM SCOPE.....	5
1.2. THE OPEN RESEARCH QUESTION.....	5
1.3. APPROACH.....	6
1.4. EXPECTED RESULTS AND IMPACT.....	7
1.5. ORGANISATION OF THIS REPORT.....	7
2. BACKGROUND.....	8
2.1. MODEL COMPRESSION: AN OVERVIEW OF FIVE PILLARS.....	8
2.2. WEIGHT SHARING IN DEPTH.....	11
2.3. THE RESEARCH GAP.....	15
2.4. ENGINEERING STANDARDS.....	16
2.5. COMPARISON WITH THE STATE OF THE ART.....	17
3. SYSTEM REQUIREMENTS.....	19
3.1. DESIGN CONSTRAINTS AND RELEVANT ENGINEERING STANDARDS.....	19
3.2. FUNCTIONAL REQUIREMENTS.....	20
3.3. NON-FUNCTIONAL REQUIREMENTS.....	22
3.4. GOALS AND EVALUATION CRITERIA.....	23
3.5. RISKS.....	24
4. USE CASES / SCENARIOS.....	27
5. PROJECT PLAN.....	30
5.1. PROJECT RESOURCES.....	30
5.2. WORK BREAKDOWN AND RESPONSIBILITIES.....	31
5.3. TIME PLAN.....	32
5.4. FUTURE WORK.....	34
6. REFERENCES.....	35

SUMMARY

This project investigates whether the $W+W^T$ intra-layer weight-sharing mechanism introduced in HaLViT [1] generalises from Vision Transformers to language models. The registered charter title is *Advanced Model Compression Techniques for Resource-Constrained AI Architectures*, and the specific research question locked with the advisor on 2026-06-12 is: *Does HaLViT's $W+W^T$ intra-layer weight-sharing mechanism generalise from Vision Transformers to language models?*

HaLViT demonstrated that a single weight matrix W and its transpose W^T can serve as two largely independent linear transformations within the same layer, provided a nonlinear activation separates them. For generic inputs and nonlinearities, $F(Wx)$ need not remain in the column space of W , so $W^T F(Wx)$ can act as a distinct transformation rather than a redundant projection. This observation halved the per-layer parameter count in ViT and ResNet architectures. Whether the same column-space argument survives the additional constraints of autoregressive language modelling — causal masking, rotary positional embeddings (RoPE), Grouped Query Attention (GQA), and SwiGLU activations — is an open question: HaLViT validates the mechanism only on vision models and frames it as extendable beyond ViT and ResNet to other architectures, but does not evaluate language models.

The proposed approach is a four-way controlled comparison. A small GPT-style language model in the range of approximately 10M–124M parameters (exact size to be fixed in Term 2) will be trained from scratch under a matched compute budget on WikiText-103, in four configurations: (1) no weight sharing; (2) ALBERT-style cross-layer sharing [2], which reuses a single weight matrix across all transformer layers along the depth axis; (3) HaLViT-style intra-layer sharing, which ties W and W^T within each layer along the width axis; and (4) both mechanisms simultaneously. The headline metrics are test perplexity and parameter count on WikiText-103, compared on a Pareto basis across the four arms. Training all four arms from scratch under identical hyperparameters and data eliminates confounds from pretrained weights and isolates the sharing variable causally. All core experiments are feasible on a single 12 GB consumer GPU. An optional extension to a pretrained 1.1B model under SwiGLU/RoPE/GQA is gated on cluster access and is not on the critical path.

The expected contribution is the first empirical comparison of intra-layer versus cross-layer weight sharing in the language modelling setting, together with a mathematical analysis of where the column-space argument holds or breaks. A positive result would suggest that $W+W^T$ sharing is a viable parameter-efficient architecture choice for language models; a clean negative result is equally valuable as it maps the mechanism's domain boundary.

1. INTRODUCTION

1.1. Motivation and Problem Scope

Modern transformer-based language models achieve remarkable performance, but at substantial parametric cost: most of a GPT-style model’s parameters live in the feed-forward (FFN) and attention projection matrices of each layer. Deploying, fine-tuning, or even training such models in resource-constrained settings — academic labs without large clusters, edge hardware, or rapid iteration on consumer GPUs — demands architectural strategies that cut this parameter burden without proportionate loss in capability.

Model compression research has produced five complementary families: pruning [3], [4], quantization [5], [6], weight sharing [2], knowledge distillation, and low-rank/tensor decomposition, increasingly combined rather than applied in isolation [7], [8]. Weight sharing is structurally distinctive: rather than removing or approximating parameters after training, it constrains the architecture so that fewer distinct parameters are allocated from the outset. ALBERT [2] is the canonical language-domain instance, reusing one weight matrix across all layers (cross-layer, depth-axis sharing) for an 18× parameter reduction relative to BERT-Large. HaLViT [1] introduced a qualitatively different geometry — sharing W and W^T within a single layer (width axis) — validated on Vision Transformers and ResNets; its discussion frames the mechanism as extendable beyond those vision architectures, though it does not evaluate language models. Whether this intra-layer mechanism survives translation to autoregressive language modelling is not obvious, and it is this uncertainty the project sets out to resolve.

1.2. The Open Research Question

HaLViT’s mechanism is geometric: a nonlinearity F placed between W and its transpose breaks the linear dependence of the two projections, so $W^T \cdot F(W \cdot x)$ is a genuinely independent transformation rather than a redundant inverse — collapsing two matrices into one and halving the block’s parameters. (The mechanism is derived in full in the Background section.) Autoregressive language models differ from the Vision Transformers HaLViT validated on in several structurally significant ways, and the project investigates two questions:

1. **Does the column-space argument remain valid under the architectural constraints of language models?** Causal masking, rotary positional embeddings (RoPE), Grouped Query Attention (GQA), and SwiGLU activations each stress the argument differently. It is strongest for the FFN path and demonstrably weaker for the attention path, where the nonlinearity is indirect — an asymmetry the experiments are designed to expose.
2. **Can intra-layer and cross-layer sharing stack beneficially?** HaLViT shares along the width axis, ALBERT along the depth axis. The two are geometrically orthogonal and the literature treats them separately, but no controlled study has measured them side by side or in combination on a language model.

A clean answer to either question — positive or negative — is a contribution, because both define the boundary of an architecture-level compression technique so far demonstrated only in the vision domain.

1.3. Approach

The project addresses both questions through a four-way controlled experiment: a small GPT-style model ($\approx 10\text{M}$ – 124M parameters, exact size fixed in Term 2) trained from scratch in four configurations — no sharing; ALBERT-style cross-layer sharing [2]; HaLViT-style $W+W^T$ intra-layer sharing [1]; and both combined — under matched hyperparameters, the same data (WikiText-103), and the same wall-clock compute budget. The headline result is test perplexity reported against parameter count on WikiText-103, compared on a Pareto basis across the four arms. Training from scratch is deliberate: it isolates the sharing variable cleanly and avoids confounds from pretrained weight distributions. A hard charter constraint requires any sharing variant to stay within 2% perplexity degradation of the unshared baseline to count as viable.

The pipeline will be implemented in PyTorch with reproducibility enforced through fixed seeds and released configurations; parameter counts, model sizes, and GFLOPs are reported per ITU-T F.748.11, with terminology following ISO/IEC 22989:2022. All core experiments run on a single 12 GB consumer GPU, making the approach independent of cluster allocations. An optional stretch — extending $W+W^T$ sharing to a pretrained 1.1B model (TinyLlama) under SwiGLU/RoPE/GQA — is pursued only if SP4CING/ITU-UHEM cluster access is granted and is not load-bearing.

1.4. Expected Results and Impact

The primary deliverable is a four-way comparison table reporting test perplexity and parameter count (with model size and FLOPs) for each configuration. A positive result quantifies the achievable parameter reduction (nominally $\approx 50\%$ of FFN and attention parameters per layer) and establishes $W+W^T$ sharing as a viable parameter-efficient choice alongside — and potentially superior to — cross-layer sharing for language modelling. A negative result is equally useful: a mechanistic account of where and why the column-space argument breaks under causal masking, RoPE, or GQA, narrowing the technique’s known applicability domain.

The work provides the first direct empirical comparison of the two major weight-sharing axes in a controlled language modelling setting, extending the known interactions between structural design choices and compression mechanisms. For resource-constrained NLP — edge inference, low-resource languages, on-device personalisation — a positive result offers an architecture-level compression route that sidesteps the tooling and calibration costs of post-training quantization or pruning. The research community is the primary stakeholder; no deployment artefact or end-user application is in scope.

1.5. Organisation of This Report

The remainder of this report is structured as follows. The **Background** section situates the project within the model compression literature, reviewing the five compression pillars and the prior weight-sharing work most directly relevant to the thesis contribution. The **System Requirements** section specifies the functional and non-functional requirements of the experimental research pipeline and presents the risk register. The **Use Cases / Scenarios** section describes the researcher-facing interaction modes with the pipeline. The **Project Plan** section presents the resources, work breakdown, and phased implementation schedule. A **References** section closes the document.

2. BACKGROUND

This section reviews the literature directly relevant to this project’s research question: whether HaLViT’s $W+W^T$ intra-layer weight-sharing mechanism generalises from Vision Transformers to language models. The section first surveys the five established pillars of model compression that frame this question. It then examines weight sharing in depth, distinguishing cross-layer and intra-layer variants and situating HaLViT precisely within that landscape. The identified gap in the literature — the absence of any empirical comparison between intra-layer and cross-layer sharing in a language modelling setting — motivates the four-way controlled experiment described in the System Requirements section. The section closes with the engineering standards that govern measurement and terminology in this work, and with a brief comparative table positioning the proposed design against the state of the art.

2.1. Model Compression: An Overview of Five Pillars

The growth of neural network models has outpaced the rate at which inference hardware has become cheaper and more energy-efficient. A sustained research effort has consequently emerged around reducing the resource cost of deploying trained networks without unacceptable accuracy loss. That effort has converged on five broadly complementary families of technique: pruning, quantization, weight sharing, knowledge distillation, and tensor decomposition. Each family attacks a different facet of over-parameterisation, and contemporary work increasingly combines them. A survey by Cheng et al. [7] provides a foundational taxonomy of the first three families; a later survey by Zhu et al. [8] updates the picture specifically for large language models. The subsections below treat each pillar in turn, with weight sharing receiving the most extended treatment because it is the mechanism under study.

2.1.1. Pruning

Pruning removes parameters whose contribution to model output is below some threshold. LeCun, Denker, and Solla [9] established the second-order saliency criterion — Optimal Brain Damage — which uses the diagonal Hessian to estimate how much each weight contributes to the training loss; this principle underlies most modern one-shot pruning methods. Han et al. [3] operationalised iterative magnitude pruning into a practical three-step pipeline (train connectivity, prune below a per-layer threshold, retrain), achieving nine-fold compression on AlexNet

with no accuracy loss. Han, Mao, and Dally extended this into the Deep Compression pipeline [10], combining pruning with trained quantization and Huffman coding to achieve 35× and 49× compression on AlexNet and VGG-16 respectively. Frankle and Carlin [11] provided a theoretical account of why iterative magnitude pruning works: the Lottery Ticket Hypothesis states that dense networks contain sparse subnetworks — winning tickets — that, when trained from their original initialisation, match the full network at ten to twenty per cent of its parameters. The practical boundary for large-scale pruning without retraining was pushed by SparseGPT [4], which prunes massive language models in one shot via a second-order Hessian approximation, and Wanda [12], which matches it using only weight magnitude times input-activation norm, with no weight update. Structural and blockwise variants — LLM-Pruner [13] and BESA [14] — extend pruning to learned per-layer sparsity.

For the present project, the interaction between pruning and weight sharing is noted in the anchor paper [1], whose discussion raises combining HaLViT with other compression techniques. Because HaLViT ties a layer’s two weight matrices to a single physical tensor W , a pruning mask M applied to the stored W is implicitly reused through the transposed path as M^T . The composability question is therefore whether pruning one stored tensor harms the two distinct roles it plays — not whether the mask must be symmetric, which is not even well-defined for the rectangular W of an FFN. The project does not implement pruning in its core four-way comparison; this composability question informs the framing of the research gap and the Future Work discussion.

2.1.2. Quantization

Quantization reduces the numerical precision of weights and/or activations rather than changing architecture, trading bounded accuracy for large memory and arithmetic savings. Jacob et al. [15] established the canonical INT8 quantisation-aware training scheme with fake-quantisation nodes and straight-through gradient estimation; Gholami et al. [16] survey the broader design space (uniformity, granularity, calibration, bit-width). GPTQ [5] and AWQ [6] are representative post-training methods for large language models, while LLM.int8() [17] shows that systematic activation outliers become a dominant obstacle only at large scale — around 6.7 B parameters and above. Because this project isolates architectural

weight sharing, quantization is out of scope for the core experiment and is treated only as future composability work; no claim is made about post-training-quantization losslessness at the target model scale.

2.1.3. Knowledge Distillation

Knowledge distillation (KD) trains a smaller student on the softened output distribution of a larger teacher, transferring generalisation ability at reduced parameter cost. Kerpici et al. [18] demonstrate layer-wise KD on speech transformers ($\approx 75\%$ size reduction at under 1% accuracy loss), and Wu et al. [19] survey federated-edge KD variants. KD is orthogonal to weight sharing and out of scope for the core experiment; it is noted only as a compatible post-training step, since an unshared model could distil into a $W+W^T$ -shared student.

2.1.4. Tensor Decomposition and Low-Rank Factorisation

Tensor decomposition factorises weight matrices into products of smaller matrices, reducing both parameter count and floating-point operations per forward pass. For 2D weight matrices the standard tool is Singular Value Decomposition (SVD): $W \approx U\Sigma V^T$, truncated to rank r . Tucker decomposition generalises this to convolutional kernels; CP decomposition produces very compact representations but is numerically unstable.

Basis Sharing [20] is the most directly relevant recent advance: adjacent transformer layers are found to share a common left-singular-vector basis, and this structure is exploited by concatenating the weight matrices of n adjacent layers, applying activation-aware scaling (Cholesky of the stacked input covariance), and decomposing into a shared basis B plus per-layer coefficient matrices $C^{(i)}$. The result — $W^{(i)} \approx B \cdot C^{(i)}$ — strictly outperforms all per-layer SVD baselines at 20–50% compression and dominates ALBERT-style identical-weight cross-layer sharing on perplexity at a fraction of the training cost. Kalle et al. [21] extend SVD compression to use a distribution-aware (function-space) error norm, demonstrating that minimising the Frobenius norm of the output-error matrix — weighted by the input activation covariance — substantially outperforms weight-space Frobenius minimisation on ResNets.

Tensor decomposition is mathematically adjacent to the $W+W^T$ mechanism: both exploit the algebraic structure of weight matrices. Whether HaLViT’s sharing is equivalent to a constrained SVD, and whether SVD-based initialisation could stabilise

training of $W+W^T$ -shared language models, remains an open theoretical question not resolved in this project.

2.2. Weight Sharing in Depth

Weight sharing is the pillar most central to this project. The literature contains two structurally distinct families, operating on orthogonal axes of the parameter tensor: cross-layer sharing (depth axis) and intra-layer sharing (width axis). This section describes both families and then situates HaLViT’s mechanism precisely.

2.2.1. Cross-Layer Weight Sharing

Cross-layer sharing ties the same weight matrix across multiple transformer blocks. The canonical instance is ALBERT [2], which applies a single set of weight matrices to every one of L transformer layers. Because the embedding matrix would otherwise dominate parameter count, ALBERT introduces factorised embedding parameterisation, decoupling the vocabulary embedding dimension E from the hidden dimension H ($E \ll H$), reducing the embedding matrix from $V \times H$ to $V \times E + E \times H$. The combined effect is dramatic compression: ALBERT-large achieves 18 M parameters against BERT-large’s 334 M — an $18\times$ reduction — while losing only 1.5 average GLUE points for all-layer sharing. An important ablation in ALBERT reveals an asymmetry: sharing only the attention weights costs virtually nothing (+0.1 GLUE average), while sharing the FFN weights is where accuracy loss materialises (−1.4 average). This decomposition of the accuracy budget between attention and FFN is directly relevant to the proposed four-way experiment. Notably, ALBERT’s layer representations do not converge as training progresses; they oscillate in L2 distance and cosine similarity, indicating that the shared-weight solution space is structurally different from the converging representations of standard BERT — the network learns to cycle through transformations rather than learn L independent ones.

ALBERT’s all-layer sharing, however, catastrophically degrades generative quality. Reid, Marrese-Taylor, and Matsuo [22] demonstrate this concretely on WMT’14 English-German machine translation: all-layer ALBERT-style sharing collapses BLEU from 27.3 to 14.3. Their diagnosis is that the first and last transformer layers encode task-specific input and output representations that cannot be shared without severe quality loss. Their remedy, the Sandwich architecture, preserves independent weights for the first and last layers while sharing only the middle layers; this recovers full BLEU at 38 M parameters versus the 65 M baseline. The Subformer further

introduces Self-Attentive Factorised Embeddings (SAFE) for seq2seq parameter efficiency. On WikiText-103 language modelling, the 122 M Subformer achieves 19.90 perplexity versus 24.03 for a 151 M Transformer-XL, while converging in 65% fewer iterations [22].

More recent work has moved beyond identical-weight cross-layer sharing to shared-basis sharing. Basis Sharing [20] demonstrates that a shared SVD basis with per-layer coefficients — $W^{(i)} \approx B \cdot C^{(i)}$ — strictly dominates identical-weight sharing (ALBERT / Dynamic Tying) for post-training compression, because the unique per-layer coefficients recover the expressiveness sacrificed by forcing identical weights. A model three times smaller under Basis Sharing achieves lower WikiText-2 perplexity than a Dynamic Tying model trained for 13.75 hours, in 26 seconds of calibration. This suggests identical-weight sharing is a weaker standalone post-training compression baseline than shared-basis methods, while remaining architecturally valid — and a useful controlled experimental arm — for studying the effect of cross-layer sharing on a language model’s representational dynamics.

The empirical study of cross-layer sharing topology has been extended by Lin et al. [23], who systematically enumerate two-dimensional sharing configurations — varying the sharing mapping (sequential, strided, pyramid, random) and the sharing distribution (uniform, front, middle, back) — in isotropic architectures including ViT and DeiT. Their centred kernel alignment (CKA) analysis shows that middle transformer layers exhibit the highest cross-layer representational similarity and are therefore the best candidates for sharing, consistent with the Sandwich finding. A key practical observation from this work is that normalization parameters can be sensitive to sharing; accordingly, in our GPT-style implementation LayerNorm parameters are kept independent across layers unless explicitly ablated. Additionally, initialising a shared-weight model by fusing the pretrained weight matrices via a channel-weighted mean, rather than randomly reinitialising, recovers 0.5–1.3 percentage points of accuracy on DeiT-S at 2× compression.

2.2.2. Intra-Layer Weight Sharing

Intra-layer sharing ties parameters within a single layer’s computation rather than across layers. Early work explored this via hash-based assignment: HashedNets [24]

map each connection to a shared scalar bucket using a hash function, with a sign factor that prevents bias from hash collisions. Soft Weight-Sharing [25] takes a Bayesian approach, learning a Gaussian mixture model over all weights and snapping each weight to its cluster centroid after training, unifying pruning and quantization in one procedure.

HaLViT [1] occupies a distinct and principled position within intra-layer sharing. The core insight is algebraic: for a weight matrix $W \in \mathbb{R}^{m \times n}$ applied to input x , the output $y = Wx$ lies in the column space of W . After a nonlinear activation $F(\cdot)$, $F(Wx)$ generically leaves that column space — for typical activations and inputs it is no longer expressible as a linear combination of the columns of W . $W^T \cdot F(Wx)$ is then a largely independent transformation rather than a redundant linear reuse of W , so two separate weight matrices can collapse to one — halving per-layer parameters while preserving the independence of the two operations.

In the Multi-Head Attention (MHA) of a Vision Transformer, HaLViT applies this as follows: a single matrix W_{kv} generates keys via $W_{kv} \cdot x$ and values via $W_{kv}^T \cdot x$; a single matrix W_q generates queries via $W_q \cdot x$ and the output projection via $W_q^T \cdot \hat{x}$. Four attention matrices collapse to two. In the FFN block, the standard two-matrix sequence W_1, W_2 in $\text{FFN}(x) = W_2 \cdot F(W_1 \cdot x + b_1) + b_2$ is replaced by a single matrix: $\text{FFN}(x) = W^T \cdot F(W \cdot x + b_1) + b_2$. The parameter count of every attention block and every FFN block is halved.

Evaluated on ImageNet-1K, HaLViT-T² (11.1 M parameters, 600-epoch training) matches PVTv2-B1 (14.0 M, 78.7% top-1) at 78.8%, using 2.9 M fewer parameters [1]. A ResNet50 variant with HaLViT sharing achieves 75.1% top-1 at 13.4 M parameters versus 76.1% for the 25.6 M standard ResNet50 — approximately two-fold parameter reduction at one percentage-point accuracy cost [1]. HaLViT’s ablation of extreme cross-layer stacking (HaLViT*, all layers shared across depth at 9 M parameters) collapses top-1 accuracy to 67.6%, demonstrating that the intra-layer $W+W^T$ mechanism — not cross-layer parameter reduction — is the source of HaLViT’s competitive efficiency. This result is noteworthy: it shows that stacking cross-layer sharing naively on top of intra-layer sharing degrades performance sharply in the vision domain, while either mechanism alone is safe at moderate compression

ratios. This vision-domain ablation does not foreclose the stacking hypothesis in language models, where the two sharing axes may interact differently; the proposed four-way comparison includes a combined-scheme arm precisely to test this empirically.

A comparison of the two axes of sharing at matched compression ratios illuminates a structural symmetry in their accuracy budgets. Lin et al. [23] report that cross-layer sharing at $2\times$ compression on DeiT-S incurs -1.08% top-1 accuracy loss with weight fusion. HaLViT’s intra-layer $W+W^T$ sharing at $2\times$ per layer incurs approximately -1.1% top-1 on the same ImageNet benchmark [1]. The similarity of these figures suggests that the accuracy cost at a given compression ratio may reflect a floor set by task difficulty rather than by the sharing mechanism itself — a hypothesis the proposed four-way experiment will probe in the language modelling domain.

2.2.3. Structural Comparison: Cross-Layer versus Intra-Layer Sharing

The two families compress different dimensions of the parameter tensor and are therefore mathematically orthogonal. ALBERT shares one weight matrix across all L transformer layers (sharing along the depth axis): compression scales linearly with L , reaching $12\times$ or $18\times$ in the ALBERT-base and ALBERT-large configurations. HaLViT shares W and W^T within a single layer (sharing along the width axis): compression is a fixed $2\times$ per layer regardless of depth. The two mechanisms address different parts of the over-parameterisation problem and, as the wiki analysis establishes, they can in principle be stacked.

The table below summarises the key structural properties of the four sharing variants surveyed.

Scheme	Sharing axis	Compression ratio	Mathematical justification	Domain	Citation
ALBERT	Depth (cross-layer, identical)	$\sim 9\text{--}18\times$ ($\sim 18\times$ total for ALBERT-large vs BERT-large; varies with L and embedding factorisation)	Empirical layer redundancy	NLP encoder	[2]
Subformer / Sandwich	Depth (middle layers only)	$\sim 1.7\times$	Empirical + first/last independence	NLP generative	[22]

Scheme	Sharing axis	Compression ratio	Mathematical justification	Domain	Citation
Basis Sharing	Depth (shared basis + unique coefficients)	20–50%	SVD cross-layer structure	NLP decoder	[20]
HaLViT	Width (intra-layer $W+W^T$)	$\sim 2\times$ per layer	Column-space independence after nonlinearity	Vision (ViT, ResNet)	[1]

The combined scheme — applying HaLViT intra-layer sharing within each layer while also applying ALBERT cross-layer sharing across layers — would reduce the affected weight matrices of an L -layer model from $2L$ independent tensors to a single shared tensor. For those two-matrix subblocks this is an idealised $2L$ reduction — about $24\times$ for $L = 12$ — before accounting for embeddings, normalization parameters, biases, output heads, and any projections left unshared, all of which lower the whole-model figure. This is consistent with the $\approx 1/(2L)$ parameter fraction stated for the combined arm in the Goals section.

2.3. The Research Gap

No published work has tested whether HaLViT’s $W+W^T$ intra-layer sharing mechanism transfers to language models. HaLViT itself is restricted to Vision Transformers and ResNets [1]; its Discussion frames the mechanism as a versatile strategy extendable beyond those vision architectures to other domains, but provides no language-model experiments. Testing autoregressive language models is therefore a natural extension this project identifies, not a result HaLViT demonstrates. The ALBERT paper [2] and the Subformer [22] thoroughly characterise cross-layer sharing in both encoder and decoder language models, but neither applies or compares intra-layer sharing. Basis Sharing [20] introduces a sophisticated SVD-based cross-layer method for large language models, but its sharing axis is still the depth axis — it does not investigate intra-layer sharing in the language modelling context.

Three distinct sub-questions have not been addressed in the literature:

1. **Architectural compatibility:** Does the column-space independence argument — which HaLViT uses to justify W and W^T as independent

transformations — survive the components of modern autoregressive language models, specifically causal masking, rotary positional embeddings (RoPE), grouped-query attention (GQA), and SwiGLU activations? For the standard FFN (BERT/GPT-2 style), the argument transfers directly: GeLU breaks the column space constraint identically to the ViT case. For the SwiGLU FFN (LLaMA/Mistral style), the intermediate representation after SiLU gating also exits the column space of W_{up} , meaning $W_{\text{down}} = W_{\text{up}}^T$ is justified — though this reduces three matrices to two rather than two to one, yielding a 33% rather than 50% FFN reduction. The attention path is the weakest link: K and V are linear projections of the same input without an intervening nonlinearity, so HaLViT relies on the downstream Softmax attention providing sufficient nonlinear mixing. Whether this suffices under causal masking and RoPE is an open empirical question.

2. **Comparative performance:** How does intra-layer sharing (HaLViT) compare to cross-layer sharing (ALBERT) in the language modelling domain when evaluated on a matched-parameter or matched-compute basis? No prior work measures both mechanisms on the same benchmark with the same evaluation protocol.
3. **Composability:** Whether intra-layer and cross-layer sharing can be stacked without destructive interference — an outcome that the HaLViT ablation of extreme stacking warns against [1] — has not been tested in language models.

This project addresses all three sub-questions through a controlled four-way empirical comparison described in the System Requirements section.

2.4. Engineering Standards

Two engineering standards govern the measurement methodology and terminology used throughout this report.

ITU-T F.748.11 defines metrics and evaluation methods for DNN processor and accelerator performance. This standard provides the formal basis for reporting compression results in terms of parameter count, model size in megabytes, and floating-point operations (FLOPs) — the three quantifiable compression axes mandated by the project charter. The present project adopts the parameter count and FLOPs accounting conventions of ITU-T F.748.11 for all four experimental arms.

ISO/IEC 22989:2022 establishes AI concepts and terminology for the broader systems context. Its definitions of “model compression,” “parameter sharing,” and “inference efficiency” are used throughout this report for terminological consistency. The report’s use of “weight sharing” rather than “weight tying” follows the conventions of the reviewed literature and is consistent with ISO/IEC 22989:2022’s vocabulary.

Reproducibility norms derived from the NeurIPS Reproducibility Checklist are adopted as a best-practice complement to these standards: all experiments will be executed with fixed random seeds, released configuration files, and publicly available datasets (WikiText-103 for the core comparison), enabling independent verification of reported perplexity values.

2.5. Comparison with the State of the Art

The table below positions the proposed experimental design relative to the four most directly comparable lines of work in the weight-sharing literature. The comparison axis is the combination of sharing mechanism, model domain, evaluation metric, and whether both sharing axes are studied simultaneously.

Work	Sharing mechanism	Domain	Metric	Both axes studied?
ALBERT [2]	Cross-layer (identical W across L layers)	NLP encoder	GLUE, MLM	No — depth axis only
Subformer [22]	Cross-layer (Sandwich, middle layers)	NLP generative	BLEU, PPL	No — depth axis only
Basis Sharing [20]	Cross-layer (shared basis + unique coefficients)	NLP decoder (LLMs)	PPL	No — depth axis only
HaLViT [1]	Intra-layer ($W+W^T$ within one layer)	Vision (ViT, ResNet)	ImageNet top-1	No — width axis only
This project	Both: intra-layer (HaLViT) + cross-layer (ALBERT)	NLP: small GPT-style LM	Perplexity-per-parameter, WikiText-103	Yes — four-way controlled comparison

The proposed design is the first to place the two orthogonal sharing axes in direct empirical comparison within the language modelling domain, with identical model architecture, training data, and compute budget across all four arms. The result of this comparison — whether intra-layer sharing outperforms, underperforms, or is orthogonally composable with cross-layer sharing — is the primary contribution of the project, as discussed in detail in the System Requirements section.

3. SYSTEM REQUIREMENTS

This section documents the design constraints, functional requirements, non-functional requirements, evaluation criteria, and risk register for the experimental pipeline described in this report. Because the project is a **research investigation** — not a deployed product — all requirements are stated in terms of what the experimental pipeline shall implement, what quality attributes the research artefacts shall satisfy, and what numerical outcomes will constitute success. There are no requirements concerning uptime, concurrent users, latency, or user-facing interfaces.

3.1. Design Constraints and Relevant Engineering Standards

3.1.1. Design Constraints

The experimental pipeline is subject to the following constraints, each derived from the project charter, the available compute resources, or the methodological requirements of a controlled comparison.

C1 — Compute hardware. All four experimental arms (no sharing, ALBERT-style cross-layer sharing, HaLViT-style intra-layer $W+W^T$ sharing, and the combined scheme) shall be trainable to convergence on a single NVIDIA RTX 4070 GPU (12 GB) without cluster access. This constraint motivates the choice of a small from-scratch GPT-style model as the primary target.

C2 — Perplexity-degradation envelope. The charter imposes a hard accuracy constraint: any shared model presented as “viable” must achieve WikiText-103 test perplexity no more than 2% above the uncompressed baseline. A configuration that exceeds the envelope is still reported — as a result characterising the compression limit of that scheme.

C3 — Framework and reproducibility. The pipeline shall be implemented in PyTorch, and every experiment shall be reproducible from a fixed random seed and a released configuration file. The dataset (WikiText-103, optionally an OpenWebText subset) is a public benchmark, ensuring external reproducibility.

C4 — Model scale. The primary target is a small language model in the $\approx 10\text{M}$ – 124M parameter range — the range within which four full from-scratch training runs fit the

single-GPU budget and the Term 2 timeline. The exact size is left as a range to be locked in Term 2 once per-arm training time on the 4070 is confirmed.

C5 — Optional stretch scope. Extension to TinyLlama 1.1B (SwiGLU, RoPE, GQA) is an optional stretch contingent on SP4CING/ITU-UHEM cluster access. It is **not** on the critical path; the thesis stands on the small-LM four-way comparison alone.

C6 — Late-start timeline. Implementation begins in Term 2 (Fall 2026). This Part-1 report is a planning and requirements document; no experimental results exist at submission time.

3.1.2. Relevant Engineering Standards

ITU-T F.748.11 (*Metrics and evaluation methods for DNN processor and accelerator performance*) defines standardised metrics — throughput, memory footprint, computational cost — for deep neural network processors. The pipeline complies by reporting all compression results in its terms: parameter count (millions), model size (MB), and floating-point operations (GFLOPs) per arm. Reporting in these standardised terms keeps results comparable to the broader compression literature.

ISO/IEC 22989:2022 (*Artificial intelligence — Concepts and terminology*) establishes a common vocabulary for AI systems. This project adopts its definitions of “model compression,” “parameter sharing,” and “parameter efficiency” throughout, ensuring the distinction between intra-layer sharing (HaLViT-style $W+W^T$) and cross-layer sharing (ALBERT-style [2]) is stated unambiguously.

Additionally, the NeurIPS/ML reproducibility checklist is adopted as a **best-practice norm** (not a formal standard): fixed random seeds, public dataset usage, released hyperparameter configurations, and a description of compute used.

3.2. Functional Requirements

The following numbered requirements specify what the experimental pipeline **shall** implement and produce. They are ordered by dependency: earlier requirements must be satisfied before later ones can be executed.

1. **FR-1 — $W+W^T$ intra-layer sharing module.** The pipeline shall provide a PyTorch module that replaces a standard FFN block ($W_2 \cdot F(W_1 \cdot x)$) with a HaLViT-style block ($W^T \cdot F(W \cdot x)$), controlled by a boolean flag. When disabled the

module shall be identical to a standard FFN. W shall be the only stored parameter; W^T shall be derived in the forward pass.

2. **FR-2 — ALBERT-style cross-layer sharing module.** The pipeline shall tie the weight matrices of all L transformer layers to a single shared copy, consistent with ALBERT [2], configurable independently of FR-1 so the two mechanisms can be combined or used in isolation.
3. **FR-3 — Four-arm training harness.** The pipeline shall train four variants from a single configuration file — (i) baseline, (ii) ALBERT-style only, (iii) HaLViT-style only, (iv) both — under a matched compute budget: identical depth/width, hyperparameters, data order (fixed seed), and number of gradient steps.
4. **FR-4 — WikiText-103 training and evaluation.** The pipeline shall load, tokenise, and iterate over WikiText-103 for training, and evaluate each trained arm on its test set, reporting token-level perplexity as the primary metric. The data pipeline shall be deterministic given the fixed seed.
5. **FR-5 — Compression metric computation.** For each arm the pipeline shall record total parameter count (millions), model size (MB, FP32 and optionally BF16), and estimated GFLOPs per forward pass, in compliance with ITU-T F.748.11 conventions.
6. **FR-6 — Four-way comparison table.** The pipeline shall emit a structured table with one row per arm — sharing scheme, parameter count, model size, GFLOPs, and WikiText-103 test perplexity. This table is the primary deliverable and central evidence for the research question.
7. **FR-7 — Gradient and correctness checks.** The pipeline shall include automated checks verifying that the $W+W^T$ module accumulates gradients correctly through the transpose, that cross-layer sharing does not silently create independent parameter copies, and that reported parameter counts reflect unique stored parameters. These shall run as a standalone validation script before any full training run.
8. **FR-8 — Per-sublayer ablation capability.** The pipeline shall support enabling or disabling $W+W^T$ sharing independently for the attention and FFN sublayers, enabling an ablation that identifies which sublayer tolerates sharing better — addressing the FFN/attention asymmetry discussed in the Background section.
9. **FR-9 — (Stretch) SwiGLU/RoPE/GQA extension.** Contingent on cluster access, the pipeline shall be extendable to apply $W+W^T$ sharing to the SwiGLU

FFN of TinyLlama 1.1B ($W_{\text{down}} = W_{\text{up}}^T$), where three matrices collapse to two for an expected $\approx 33\%$ FFN reduction rather than 50%. This requirement is optional and does not gate the core result.

3.3. Non-Functional Requirements

The following requirements specify measurable quality attributes that the pipeline and its outputs shall satisfy. None concern deployment, uptime, or user-facing performance.

1. **NFR-1 — Reproducibility.** Every run shall be reproducible from a single configuration file (model size, sharing scheme, seed, dataset split, hyperparameters). Given the same configuration, hardware, and pinned software environment, perplexity results shall agree within floating-point tolerance. Code and configurations shall be released alongside the thesis.
2. **NFR-2 — Perplexity-degradation constraint.** Any configuration reported as “viable” shall achieve WikiText-103 test perplexity no more than 2% above the baseline trained under identical conditions (the hard charter constraint, C2). Arms exceeding it are still reported, just not recommended as viable strategies.
3. **NFR-3 — Single-GPU trainability.** Each arm shall train to convergence on a single RTX 4070 (12 GB) with peak activation memory below 11 GB. Any memory-saving trick (e.g. gradient checkpointing) shall be applied identically across all arms to preserve comparability.
4. **NFR-4 — Bounded per-arm training time.** Each arm shall complete within a wall-clock budget allowing all four to be trained and evaluated inside the Term 2 window (indicative upper bound ≈ 48 h per arm on the 4070; confirmed once model size is locked).
5. **NFR-5 — Numerical correctness of sharing.** The $W+W^T$ implementation shall pass the FR-7 gradient checks: no detached tensors, stop-gradient operations, or silent parameter copies, and parameter counts shall reflect unique stored parameters only.
6. **NFR-6 — Code modularity and maintainability.** The sharing modules (FR-1, FR-2) shall be separated from the training harness (FR-3) via a clean interface, so schemes can be tested independently and the harness extended (e.g. the TinyLlama stretch) without modifying the core loop. Configuration shall be file-based, not hard-coded.

7. **NFR-7 — Documentation of experimental conditions.** Each logged experiment shall record its full configuration, hardware, software environment, and training curve (validation perplexity per epoch), supporting the evaluation phase and any subsequent peer review.

3.4. Goals and Evaluation Criteria

The following goals are the measurable outcomes that determine whether the research questions have been answered. Each is assessed during the evaluation phase described in the Project Plan.

G1 — Four-way perplexity vs. parameter-count comparison

Establish whether HaLViT-style intra-layer sharing achieves lower perplexity at equivalent parameter count than ALBERT-style cross-layer sharing.

Evaluation Criteria:

- For each of the four arms, a (perplexity, parameter count) pair is measured on WikiText-103.
- Perplexity vs. parameter count is plotted across all arms; the arm closest to the Pareto frontier of low perplexity and low parameter count is the empirically strongest scheme.

G2 — Parameter-reduction ratios

Confirm that measured parameter counts match the analytical predictions for each sharing scheme (embeddings excluded, consistent with the Background derivation).

Evaluation Criteria:

- $W+W^T$ intra-layer sharing reduces per-layer FFN parameters by $\approx 50\%$ (a fixed $2\times$ per layer, independent of depth).
- ALBERT-style cross-layer sharing reduces transformer-block parameters to $\approx 1/L$ of baseline ($\approx 8\%$ for a 12-layer model).
- The combined scheme reduces them to $\approx 1/(2L)$ of baseline ($\approx 4\%$ for a 12-layer model).
- Measured stored counts (FR-5) are compared against these predictions; deviations larger than 1% are explained.

G3 — Perplexity-degradation gate

Verify that each arm presented as viable satisfies the $<2\%$ degradation constraint (NFR-2) relative to the no-sharing baseline.

Evaluation Criteria:

- $(\text{PPL}_{\text{arm}} - \text{PPL}_{\text{baseline}}) / \text{PPL}_{\text{baseline}}$ is computed for each arm and reported in the comparison table.
- Arms exceeding the 2% threshold are reported but not recommended as viable strategies.

G4 — Per-sublayer ablation

Determine which sublayer (attention vs. FFN) tolerates $W+W^T$ sharing better.

Evaluation Criteria:

- Perplexity is measured for sharing applied to attention only, FFN only, and both, each against the no-sharing baseline.
- The relative perplexity costs identify the more shareable sublayer. The project hypothesis — derived from the column-space argument, which is cleaner for the FFN path than for attention — is that FFN sharing is less costly than attention sharing; the ablation confirms or refutes this.

G5 — (Stretch) Architectural compatibility with SwiGLU/RoPE/GQA

If TinyLlama experiments are conducted, establish whether $W_{\text{down}} = W_{\text{up}}^T$ sharing recovers to within 2% perplexity of the pretrained baseline after fine-tuning.

Evaluation Criteria:

- Perplexity of the shared TinyLlama is compared against the unmodified TinyLlama 1.1B checkpoint on WikiText-103.

3.5. Risks

Each risk is paired with an explicit alternative or mitigation plan. Risks are ordered from highest potential impact on the core thesis to lowest.

R1 — $W+W^T$ mechanism may not transfer under causal attention

The column-space argument that justifies $W+W^T$ sharing [1] is strong for FFN layers, where GeLU or SwiGLU supplies a clean nonlinearity between the two matrix applications. For attention it is weaker: Keys and Values are both linear projections of the same input, and the nonlinear mixing occurs downstream via Softmax. Under

causal masking the Softmax is applied over a strictly lower-triangular window, which may reduce the effective mixing that justifies treating W^T as independent. If the mechanism breaks, the attention arm will show disproportionate perplexity degradation.

Alternative Plan:

- Treat a clean negative result on the attention path as a valid contribution — it defines the boundary condition of the mechanism with a mechanistic explanation.
- Use the per-sublayer ablation (FR-8, G4) to isolate the effect: if the FFN arm satisfies G3 while the combined arm does not, recommend FFN-only sharing as the practical configuration.

R2 — Compute limits may prevent training at the upper end of the model-size range

A 124M-parameter model trained from scratch across four arms may exceed the practical wall-clock budget of the RTX 4070 within the Term 2 timeline, and peak activation memory may approach the 12 GB limit at large batch sizes.

Alternative Plan:

- Select the model size conservatively during the Term 2 planning sprint, after a benchmark run measuring tokens-per-second at candidate sizes.
- If the upper end is infeasible, reduce size within the range; conclusions are scale-invariant as long as all four arms train under identical conditions.
- Rely on the advisor’s “go smaller” steer (2026-06-12): matched compute budget is a more important control than absolute model size.

R3 — Training instability arising from weight sharing

Tying W and W^T throughout training may destabilise learning dynamics. Cross-layer sharing (ALBERT) is known to reduce effective per-layer capacity and can slow convergence [2]. Intra-layer sharing constrains the weight gradient to account for both forward and transpose paths, which may interact poorly with Adam’s per-parameter moment estimates if the two paths’ gradient magnitudes differ.

Alternative Plan:

- Apply a sharing warm-up schedule that enforces the $W+W^T$ constraint only after N gradient steps, letting the model initialise in a stable region first.
- Use LoRA-style enforcement (W frozen or slowly updated, low-rank adapter implementing the deviation from W^T) — the approach planned for the TinyLlama stretch (FR-9), applicable to the core model if needed.
- Reduce the learning rate for shared layers; if instability persists, report it as an empirical finding and restrict the comparison to configurations that converge stably.

R4 — Cluster access is not granted, blocking the TinyLlama stretch

The optional TinyLlama extension (FR-9) requires SP4CING/ITU-UHEM cluster access. If access is not granted, or granted too late in Term 2, the experiments cannot be conducted.

Alternative Plan:

- Treat the four-way small-LM comparison (FR-3 through FR-6) as the complete experimental programme — it is entirely independent of cluster access and sufficient for a full thesis.
- Defer the TinyLlama SwiGLU/GQA analysis to future work, while still completing the theoretical SwiGLU-compatibility analysis (a written contribution) regardless of cluster access.

R5 — Concurrent work may reduce novelty

Basis Sharing [20] applies cross-layer SVD-based sharing to LLMs and was published after the charter was signed. Further concurrent work applying intra-layer sharing to language models before submission could reduce the novelty of the empirical contribution.

Alternative Plan:

- Frame the controlled four-way comparison under a matched training budget — and the causal isolation of the sharing variable via from-scratch training — as the distinct methodological contribution, independent of whether individual mechanisms have been studied elsewhere.
- Run a literature-monitoring step at the start of Term 2 to identify concurrent work and adjust the novelty claim accordingly.

4. USE CASES / SCENARIOS

This section describes the researcher-centric scenarios that the experimental pipeline must support. Because this project is a research investigation rather than a deployed product, the “users” of the system are the researchers conducting the four-way comparison study — the two team members and, through the released configurations and reproducibility artefacts, the broader research community. Each scenario states an actor goal followed by the system behaviour the pipeline must provide.

Configure and launch a matched-budget training run

A researcher wants to select a sharing scheme — {none, ALBERT-style cross-layer, HaLViT-style intra-layer, both} — and launch a WikiText-103 training run under a compute budget identical across all four arms, so that any perplexity difference is attributable solely to the sharing mechanism.

System Behavior:

- The pipeline shall accept a sharing-configuration flag as a single parameter.
- The pipeline shall instantiate the correct architecture variant (the $W+W^T$ module or the ALBERT cross-layer binding as appropriate).
- The pipeline shall log hyperparameters and seed to a reproducible configuration file.
- The pipeline shall execute training on the RTX 4070 (12 GB) without exceeding GPU memory capacity.

Run the mathematical-validity check before committing compute

A researcher wants to verify, before any long training run, that the $W+W^T$ sharing constraint is correctly implemented and that the column-space independence argument holds numerically for the chosen activation (GeLU for the standard FFN; SwiGLU for the optional stretch).

System Behavior:

- The pipeline shall check gradient symmetry through the shared weight.
- The pipeline shall verify that the parameter count matches the theoretical prediction ($\approx 50\%$ reduction per shared FFN layer for the standard target; $\approx 33\%$ per SwiGLU FFN for the stretch).

- The pipeline shall report any numerical anomaly before the full training loop begins.
- The pipeline shall complete this check in under five minutes on the target hardware.

Read off perplexity and parameter count to compare arms

A researcher wants a standardised evaluation summary for each of the four trained variants — final WikiText-103 test perplexity, parameter count, model size, and GFLOPs — so the four-way comparison table can be built without manual post-processing.

System Behavior:

- The pipeline shall evaluate each checkpoint on the held-out test set upon training completion.
- The pipeline shall write all metrics to a structured log (JSON or CSV).
- The pipeline shall emit a consolidated comparison table as a human-readable artefact.
- The pipeline shall report all metrics in conformance with ITU-T F.748.11.

Reproduce a run from released configurations and seeds

A researcher (or a third party replicating the results) wants to re-execute any arm from a released configuration file and seed, obtaining perplexity within a negligible tolerance of the reported result, with no proprietary tool or non-public checkpoint required.

System Behavior:

- The pipeline shall store all hyperparameters, dataset-split indices, seeds, and software-version metadata in a self-contained configuration artefact committed alongside the weights.
- The pipeline shall use only publicly available datasets (WikiText-103 and, optionally, an OpenWebText subset).
- The pipeline shall be reproducible on any machine meeting the hardware specification using only the released code and standard open-source libraries.

(Stretch) Apply the sharing constraint to a pretrained TinyLlama and measure perplexity recovery

Only if SP4CING/ITU-UHEM cluster access is granted, a researcher wants to apply the $W+W^T$ constraint to a pretrained TinyLlama 1.1B checkpoint — the SwiGLU variant

($W_{\text{down}} = W_{\text{up}}^T$) and the GQA path — and fine-tune under a LoRA-enforced constraint, testing whether the mechanism transfers to a modern LLM with SwiGLU, RoPE, and GQA simultaneously active.

System Behavior:

- The pipeline shall extend the $W+W^T$ module to handle the SwiGLU three-matrix FFN (reducing W_{gate} , W_{up} , W_{down} to two free matrices).
- The pipeline shall propagate gradients correctly through the LoRA-constrained shared weight.
- The pipeline shall report perplexity recovery as a function of fine-tuning budget.
- This scenario is strictly optional and does not affect the validity of the core four-way comparison.

5. PROJECT PLAN

This project investigates whether HaLViT’s $W+W^T$ intra-layer weight-sharing mechanism [1], demonstrated for Vision Transformers, generalises to autoregressive language models. The experimental pipeline trains four architectural variants — no sharing, ALBERT-style cross-layer sharing [2], HaLViT-style intra-layer sharing, and both combined — from scratch under a matched compute budget, evaluating each on WikiText-103 perplexity at matched parameter count. The core contribution is the first controlled four-way comparison of these two orthogonal sharing axes in the language-modelling setting.

The work described in this document represents the planning and requirements phase (Term 1, Spring 2026). All implementation, experimentation, and evaluation are scheduled for Term 2 (Fall 2026). The sections below detail the resources available, the work breakdown across team members, and the schedule aligned to the system requirements established earlier in this report.

5.1. Project Resources

The following software and hardware resources are required for the core experimental work. No paid cloud services, paid datasets, or human participants are needed.

Hardware. The primary compute resource is the team’s shared desktop workstation equipped with an NVIDIA RTX 4070 GPU (12 GB VRAM, GDDR6X). This is sufficient to train all four arms of the small GPT-style from-scratch experiment, given the matched-budget design and the advisor’s direction to keep the model within the $\approx 10\text{M}$ – 124M parameter range. The optional TinyLlama stretch requires SP4CING/ITU-UHEM GPU cluster allocation; access is not assumed and will be confirmed with Prof. Töreyn in Term 2.

Software. All implementation will use PyTorch as the primary deep-learning framework. The base codebase will be derived from either nanoGPT or the HuggingFace Transformers library (selection deferred to Term 2 based on implementation convenience for the sharing modules). Standard profiling tools (torch.profiler, ptflops, or equivalent) will be used to measure parameter counts and

GFLOPs in conformance with ITU-T F.748.11. Experiment tracking will use an open-source tool (e.g., Weights & Biases free tier or MLflow local instance).

Datasets. The primary evaluation dataset is WikiText-103 (publicly available, standard language modelling benchmark). An OpenWebText subset may be used as an auxiliary training corpus if compute budget permits. No proprietary data is used.

No additional resources — volunteers, paid APIs, licensed software, or external hardware — are required for the core thesis.

5.2. Work Breakdown and Responsibilities

The project tasks are distributed across five activity categories: Design, Implementation, Testing, Documentation, and Presentation. Both team members participate in all categories; the split below reflects primary ownership of specific subtasks.

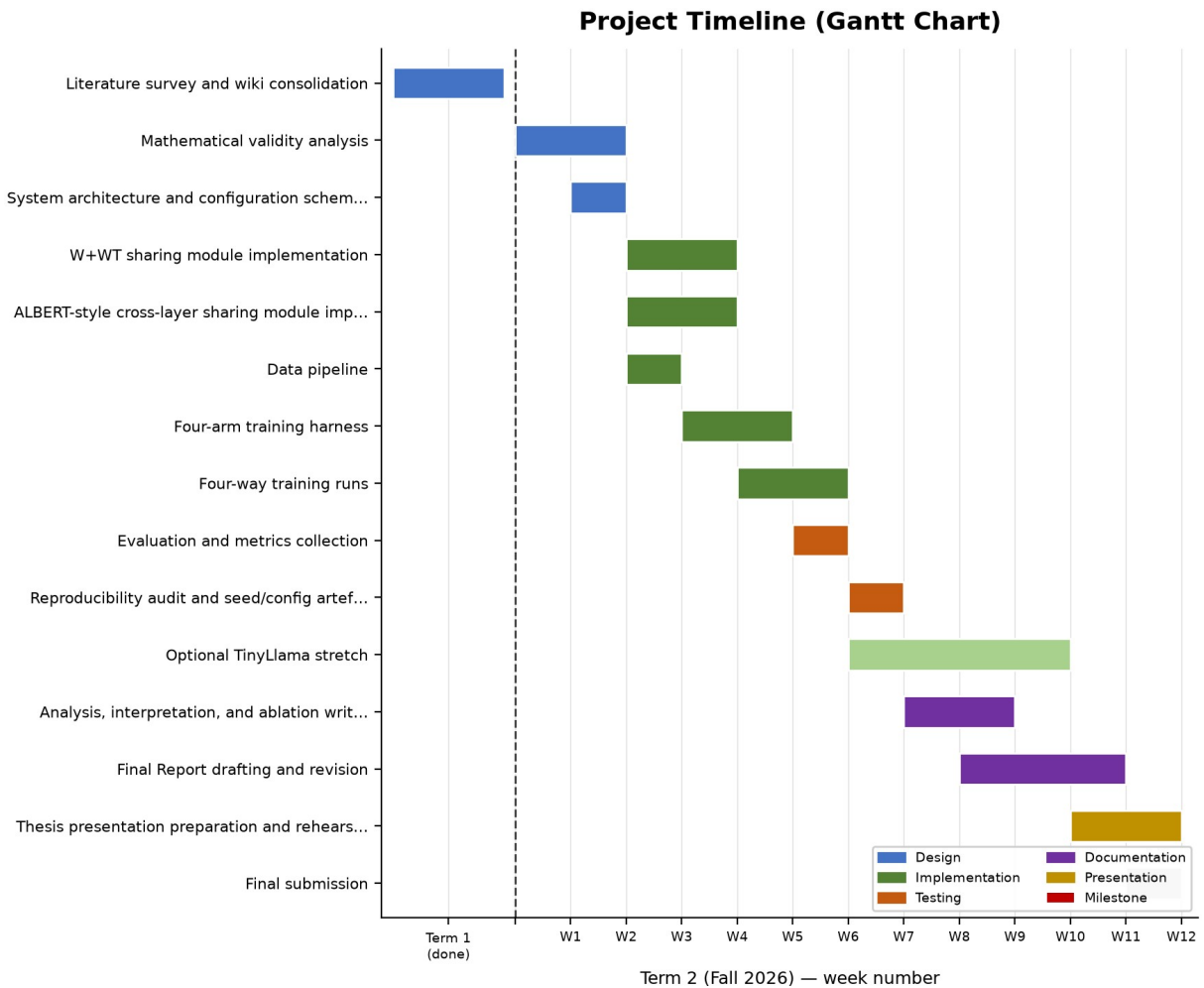
The ownership split below is a *proposed* division, to be confirmed with the advisor at the first Term 2 meeting.

Task / Subtask	Phase	Primary	Secondary
Literature review & theoretical background	Design	Both	—
Mathematical validity analysis (SwiGLU, causal attention, RoPE, GQA)	Design	Both	—
Four-way experiment protocol & hyperparameter design	Design	Both	—
$W+W^T$ intra-layer sharing module (FFN, GeLU)	Implementation	A. Düzgün	H. Sözer
Four-arm training harness (matched-budget runner, logging)	Implementation	A. Düzgün	H. Sözer
ALBERT-style cross-layer sharing module	Implementation	H. Sözer	A. Düzgün
Data pipeline (WikiText-103 tokenisation, splits, loaders)	Implementation	H. Sözer	A. Düzgün
Evaluation & metrics (perplexity, params, GFLOPs, table)	Implementation	H. Sözer	A. Düzgün
Gradient-symmetry & param-count validation (Scenario 2)	Testing	Both	—
Reproducibility audit (seeds, config, environment)	Testing	Both	—

Task / Subtask	Phase	Primary	Secondary
Four-way result verification & ablation	Testing	Both	—
Optional TinyLlama stretch (SwiGLU module, LoRA fine-tuning)	Impl. (stretch)	Both	—
Interim Report (Part 1 — this document)	Documentation	Both	—
Final Report (Part 2 — Term 2)	Documentation	Both	—
Thesis presentation preparation	Presentation	Both	—

5.3. Time Plan

The schedule below covers the full project lifecycle. Term 1 (Spring 2026) activities are planning and requirements work already partially completed; Term 2 (Fall 2026) is the implementation and evaluation period. Week numbers are relative to the start of Term 2 unless marked as Term 1.



Project timeline across Term 2 (Fall 2026), colour-coded by phase. Term 1 work is complete; the diamond marks the final-submission milestone. The table below gives the same schedule with linked requirements.

Task	Phase	Weeks	Linked Req.
Literature survey & wiki consolidation	Design	T1 done	—
Math validity analysis (SwiGLU, attention, RoPE, GQA)	Design	1–2	FR-7
System architecture & config-schema design	Design	2	NFR-6
W+W ^T sharing module (FFN)	Implementation	3–4	FR-1
ALBERT cross-layer sharing module	Implementation	3–4	FR-2
Data pipeline (WikiText-103)	Implementation	3	FR-4
Four-arm training harness (matched	Implementation	4–5	FR-3

Task	Phase	Weeks	Linked Req.
budget, logging)			
Four-way training runs (all arms)	Implementation	5–6	FR-3/4, NFR-3/4
Evaluation & metrics (perplexity, params, GFLOPs)	Testing	6	FR-5/6
Reproducibility audit & seed/config release	Testing	7	NFR-1
Optional TinyLlama stretch (SwiGLU, LoRA fine-tuning)	Impl. (stretch)	7–10	FR-9
Analysis, interpretation & ablation write-up	Documentation	8–9	—
Final Report drafting & revision	Documentation	9–11	—
Thesis presentation prep & rehearsal	Presentation	11–12	—
Final submission	—	12	—

5.4. Future Work

Several directions extend naturally from the core design. First, the model scale studied here is intentionally small ($\approx 10\text{M}$ – 124M parameters, final size confirmed in Term 2); a natural successor study would apply the same four-way protocol at the 1B–7B scale on a GPU cluster, testing whether the perplexity-versus-parameter-count trends observed at small scale hold in the large-model regime. Second, this thesis examines sharing along two axes (width and depth); a third axis — sharing across attention heads or across encoder/decoder stacks — remains unexplored. Third, the $W+W^T$ constraint is currently studied in isolation from other compression techniques; composing it with structured pruning or low-bit quantisation (building on the compatibility analysis in the project wiki) is a direct and practical extension. Fourth, if the mechanism proves effective in the language-modelling setting, the practical implication for on-device language models — where parameter reduction directly reduces model storage and peak activation memory — motivates a follow-on engineering study applying the findings to resource-constrained inference settings.

6. REFERENCES

- [1] O. C. Koyun and B. U. Töreyn, "HaLViT: Half of the Weights are Enough," CVPR Workshop (Computer Vision Foundation, Open Access), 2023.
- [2] Z. Lan, M. Chen, S. Goodman, K. Gimpel, P. Sharma, and R. Soricut, "ALBERT: A Lite BERT for Self-supervised Learning of Language Representations," arXiv:1909.11942, 2020.
- [3] S. Han, J. Pool, J. Tran, and W. J. Dally, "Learning both Weights and Connections for Efficient Neural Networks," arXiv:1506.02626, 2015.
- [4] E. Frantar and D. Alistarh, "SparseGPT: Massive Language Models Can be Accurately Pruned in One-Shot," arXiv:2301.00774, 2023.
- [5] E. Frantar, S. Ashkboos, T. Hoefler, and D. Alistarh, "GPTQ: Accurate Post-Training Quantization for Generative Pre-Trained Transformers," arXiv:2210.17323, 2023.
- [6] J. Lin et al., "AWQ: Activation-Aware Weight Quantization for On-Device LLM Compression and Acceleration," arXiv:2306.00978, 2024.
- [7] Y. Cheng, D. Wang, P. Zhou, and T. Zhang, "A Survey of Model Compression and Acceleration for Deep Neural Networks," arXiv:1710.09282, 2020.
- [8] X. Zhu, J. Li, Y. Liu, C. Ma, and W. Wang, "A Survey on Model Compression for Large Language Models," arXiv:2308.07633, 2023.
- [9] Y. LeCun, J. S. Denker, and S. A. Solla, "Optimal Brain Damage," NeurIPS 1990, 1990.
- [10] S. Han, H. Mao, and W. J. Dally, "Deep Compression: Compressing Deep Neural Networks with Pruning, Trained Quantization and Huffman Coding," arXiv:1510.00149, 2016.
- [11] J. Frankle and M. Carbin, "The Lottery Ticket Hypothesis: Finding Sparse, Trainable Neural Networks," arXiv:1803.03635, 2019.
- [12] M. Sun, Z. Liu, A. Bair, and J. Z. Kolter, "A Simple and Effective Pruning Approach for Large Language Models (Wanda)," arXiv:2306.11695, 2023.
- [13] X. Ma, G. Fang, and X. Wang, "LLM-Pruner: On the Structural Pruning of Large Language Models," arXiv:2305.11627, 2023.
- [14] P. Xu et al., "BESA: Pruning Large Language Models with Blockwise Parameter-Efficient Sparsity Allocation," arXiv:2402.16880, 2024.
- [15] B. Jacob et al., "Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference," arXiv:1712.05877, 2018.

- [16] A. Gholami, S. Kim, Z. Dong, Z. Yao, M. W. Mahoney, and K. Keutzer, "A Survey of Quantization Methods for Efficient Neural Network Inference," arXiv:2103.13630, 2021.
- [17] T. Dettmers, M. Lewis, Y. Belkada, and L. Zettlemoyer, "LLM.int8(): 8-bit Matrix Multiplication for Transformers at Scale," arXiv:2208.07339, 2022.
- [18] M. Kerpici, V. Nguyen, S. Zhang, and E. Visser, "Application of Knowledge Distillation to Multi-Task Speech Representation Learning," arXiv:2210.16611, 2023.
- [19] Z. Wu et al., "Knowledge Distillation in Federated Edge Learning: A Survey," arXiv:2301.05849, 2024.
- [20] J. Wang, Y. Chen, I. Lin, B. Li, and G. L. Zhang, "Basis Sharing: Cross-Layer Parameter Sharing for Large Language Model Compression," arXiv:2410.03765, 2024.
- [21] A. Kalle, T. Rudkiewicz, M. Ouerfelli, and M. Tamaazousti, "Distribution-Aware Tensor Decomposition for Compression of Convolutional Neural Networks," arXiv:2511.04494, 2025.
- [22] M. Reid, E. Marrese-Taylor, and Y. Matsuo, "Subformer: Exploring Weight Sharing for Parameter Efficiency in Generative Transformers," arXiv:2101.00234, 2021.
- [23] C.-Y. Lin et al., "SPIN: An Empirical Evaluation on Sharing Parameters of Isotropic Networks," arXiv:2207.10237, 2022.
- [24] W. Chen, J. T. Wilson, S. Tyree, K. Q. Weinberger, and Y. Chen, "Compressing Neural Networks with the Hashing Trick," arXiv:1504.04788, 2015.
- [25] K. Ullrich, E. Meeds, and M. Welling, "Soft Weight-Sharing for Neural Network Compression," arXiv:1702.04008, 2017.